Training a Neural Network, Part 2

We now have a clear goal: minimize the loss of the neural network. We know we can change the network's weights and biases to influence its predictions, but how do we do so in a way that decreases loss?

For simplicity, let's pretend we only one data in our dataset:

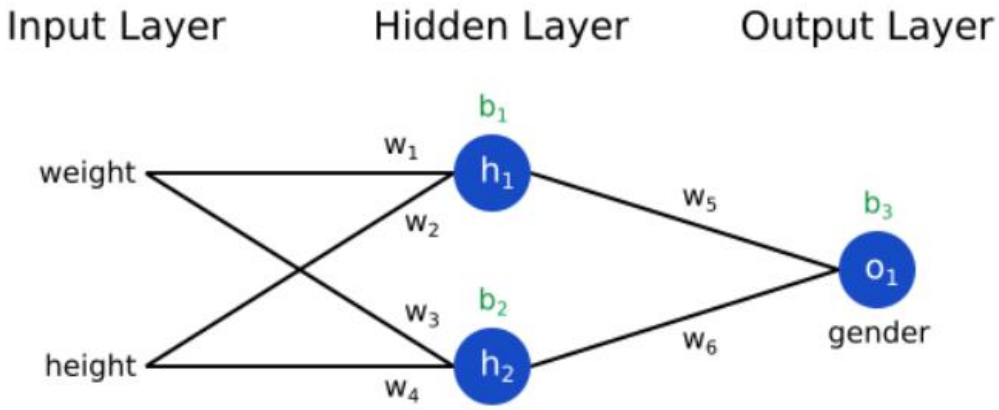
Weight (-135)	Height (-66)	Gender
-2	-1	1

Table 1

Then the mean squared error loss is:

$$\begin{aligned} \text{MSE} &= \frac{1}{1}\sum_{i=1}^{1}(y_{true} - y_{pred})^2 \\ &= (y_{true} - y_{pred})^2 \\ &= (1 - y_{pred})^2 \end{aligned}$$

Another way to think about loss is as a function of weights and biases. Let's label each weight and bias in our network:



Then, we can write loss as a multivariable function:

$$L(w_1, w_2, w_3, w_4, w_5, w_6, b_1, b_2, b_3)$$

Imagine we wanted to tweak w1. How would loss L change if we changed w1? That's a question the partial derivative can answer. How do we calculate it?

To start, let's rewrite the partial derivative in terms of $\partial Y_{pred}/\partial w1$ instead:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_{pred}} * \frac{\partial y_{pred}}{\partial w_1}$$

We can calculate $\partial L/\partial Y_{pred}$ because we computed $L= (1-Y_{pred})^2$ above:

$$\frac{\partial L}{\partial y_{pred}} = \frac{\partial (1 - y_{pred})^2}{\partial y_{pred}} = \boxed{-2(1 - y_{pred})}$$

Now, let's figure out what to do with $\partial Y_{pred}/\partial w1$. Just like before, let h1, h2, o1 be the outputs of the neurons they represent. Then

$$y_{pred} = o_1 = f(w_5 h_1 + w_6 h_2 + b_3)$$

Since w1only affects h1(not h2), we can write

$$\frac{\partial y_{pred}}{\partial w_1} = \frac{\partial y_{pred}}{\partial h_1} * \frac{\partial h_1}{\partial w_1}$$

$$\frac{\partial y_{pred}}{\partial h_1} = \boxed{w_5 * f'(w_5 h_1 + w_6 h_2 + b_3)}$$

We do the same thing for $\partial h1/\partial w1$:

$$h_1 = f(w_1 x_1 + w_2 x_2 + b_1)$$

$$\frac{\partial h_1}{\partial w_1} = \boxed{x_1 * f'(w_1 x_1 + w_2 x_2 + b_1)}$$

x1 here is weight, and x2 is height. This is the second time we've seen f'(x) (the derivate of the sigmoid function) now! Let's derive it:

$$f(x) = \frac{1}{1 + e^{-x}}$$

$$f'(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = f(x) * (1 - f(x))$$

We're done! We've managed to break down ∂L/∂w1into several parts we can calculate:

$$\boxed{\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_{pred}} * \frac{\partial y_{pred}}{\partial h_1} * \frac{\partial h_1}{\partial w_1}}$$

This system of calculating partial derivatives by working backwards is known as backpropagation, or "backprop".

Example: Calculating the Partial Derivative

Consider the single dataset in table 1. Let's initialize all the weights to 1 and all the biases to 0. If we do a feedforward pass through the network, we get:

$$\begin{aligned} h_1 &= f(w_1 x_1 + w_2 x_2 + b_1) \\ &= f(-2 + -1 + 0) \\ &= 0.0474 \end{aligned}$$

$$h_2 = f(w_3 x_1 + w_4 x_2 + b_2) = 0.0474$$

$$\begin{aligned} o_1 &= f(w_5 h_1 + w_6 h_2 + b_3) \\ &= f(0.0474 + 0.0474 + 0) \\ &= 0.524 \end{aligned}$$

The network outputs Y_{pred}= 0.524, which doesn't strongly favor Male (0) or Female (1). Let's calculate ∂L/∂w1 :

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_{pred}} * \frac{\partial y_{pred}}{\partial h_1} * \frac{\partial h_1}{\partial w_1}$$

$$\begin{aligned}\frac{\partial L}{\partial y_{pred}} &= -2(1 - y_{pred}) \\ &= -2(1 - 0.524) \\ &= -0.952\end{aligned}$$

$$\begin{aligned}\frac{\partial y_{pred}}{\partial h_1} &= w_5 * f'(w_5 h_1 + w_6 h_2 + b_3) \\ &= 1 * f'(0.0474 + 0.0474 + 0) \\ &= f(0.0948) * (1 - f(0.0948)) \\ &= 0.249\end{aligned}$$

$$\begin{aligned}\frac{\partial h_1}{\partial w_1} &= x_1 * f'(w_1 x_1 + w_2 x_2 + b_1) \\ &= -2 * f'(-2 + -1 + 0) \\ &= -2 * f(-3) * (1 - f(-3)) \\ &= -0.0904\end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial w_1} &= -0.952 * 0.249 * -0.0904 \\ &= \boxed{0.0214}\end{aligned}$$

Reminder: we derived f′(x) =f(x) ∗ (1−f(x)) for our sigmoid activation function earlier.

We did it! This tells us that if we were to increase *w1, L would increase a tiny bit as a result.*

Training: Stochastic Gradient Descent

We have all the tools we need to train a neural network now! We'll use an optimization algorithm called stochastic gradient descent (SGD) that tells us how to change our weights and biases to minimize loss. It's basically just this update equation:

$$w_1 \leftarrow w_1 - \eta \frac{\partial L}{\partial w_1}$$

η is a constant called the learning rate that controls how fast we train. All we're doing is subtracting η ∂w1/∂Lfrom w1:

- If ∂L/∂w1is positive, w1will decrease, which makes L decrease.
- If ∂L/∂w1 is negative, w1will increase, which makes L decrease.

If we do this for every weight and bias in the network, the loss will slowly decrease and our network will improve.

Our training process will look like this:

1. Choose one sample from our dataset. This is what makes it stochastic gradient descent — we only operate on one sample at a time.
2. Calculate all the partial derivatives of loss with respect to weights or biases (e.g. ∂L/∂w1, ∂L/∂w2, etc.).
3. Use the update equation to update each weight and bias.
4. Go back to step 1.